

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE CODE: CSA5101 **COURSE NAME:** Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

*I **MADHU BAVANA.B(192524103)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
MADHU BAVANA.B

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

CASE STUDY

1. A pharmaceutical research lab relies on a cryptographic hash function to guarantee integrity of shipment manifests, but a recent internal review questions whether the algorithm's security properties are still adequate. Discuss the role of hash functions in ensuring data integrity in this context, and recommend how the organization should validate its hash function's security periodically.
2. A university examination system faces recurring disputes over electronically signed approvals because it lacks a formal signature-verification and audit process. Design a complete non-repudiation framework (signing, timestamping, certificate revocation checks, and secure audit trail) that this organization should adopt going forward.
3. A smart-metering IoT platform is redesigning its network authentication architecture after a near-miss security incident and is considering strengthening its existing Kerberos-based authentication. Design a Kerberos deployment checklist (ticket lifetime policy, clock synchronization via NTP, pre-authentication, key rotation) that this organization should implement to prevent similar attacks.
4. An ed-tech learning platform is establishing a public-key infrastructure (PKI) and wants to avoid the common pitfalls of unmanaged X.509 certificate trust seen in other organizations. Design a certificate lifecycle policy covering issuance, renewal, revocation, and monitoring, and specify how automation (e.g., ACME protocol) could reduce human error in this organization.
5. A stock trading platform needs to guarantee that messages exchanged between two trusted parties have not been altered in transit and genuinely originate from the claimed sender. Design a message authentication mechanism using HMAC-SHA256 for this system, and justify the choice of underlying hash function and key length.

1. Cryptographic Hash Function for Pharmaceutical Shipment Manifests

Introduction

A pharmaceutical research laboratory uses cryptographic hash functions to ensure that shipment manifests are not modified during storage or transmission. Since shipment manifests may contain critical information such as drug names, quantities, batch numbers, destinations, and handling instructions, maintaining their integrity is essential.

Role of Hash Functions in Data Integrity

A cryptographic hash function converts a message of arbitrary length into a fixed-length hash value, also called a message digest. The laboratory can calculate the hash of the original shipment manifest and securely store or transmit the digest along with the document.

When the manifest is received, the hash is calculated again. If the newly calculated hash matches the original hash, the document is considered unchanged.

Hash functions provide important security properties:

1. **Preimage resistance** – It should be computationally infeasible to determine the original message from its hash.
2. **Second-preimage resistance** – It should be difficult to find another message having the same hash as a given message.
3. **Collision resistance** – It should be computationally infeasible to find two different messages with the same hash.
4. **Avalanche effect** – A small change in the manifest should produce a significantly different hash value.

However, a plain hash only provides integrity detection. It does **not** prove who created or sent the document because an attacker who modifies the document may also calculate a new hash. Therefore, the laboratory should combine hashing with **HMAC or digital signatures** when authenticity is required.

Periodic Security Validation

The organization should establish a formal cryptographic review process:

- Maintain an inventory of all hash algorithms currently being used.
- Check whether the algorithm has known collision, preimage, or implementation attacks.
- Prefer modern standardized algorithms such as **SHA-256 or SHA-3** for new systems.
- Avoid obsolete algorithms such as MD5 and SHA-1 for security-critical integrity protection.
- Review cryptographic standards and security advisories periodically.
- Perform independent security assessments and penetration testing.
- Verify that secure random generation, key management, and cryptographic libraries are correctly implemented.

- Test the implementation against known test vectors.
- Establish a migration plan before an algorithm becomes obsolete.

Recommended Solution

The laboratory should use **SHA-256 or SHA-3** for hashing and, where authenticity is required, use a **digital signature or HMAC** rather than relying on a bare hash.

Conclusion

Cryptographic hashing provides an efficient mechanism for detecting unauthorized modification of shipment manifests. Periodic algorithm review, implementation testing, vulnerability monitoring, and planned migration are necessary to ensure that the integrity mechanism remains secure over time.

2. Non-Repudiation Framework for a University Examination System

Introduction

An electronic examination system requires strong non-repudiation because approvals, marks, examination results, and administrative decisions may later be disputed. A complete framework should provide evidence that a particular authorized person signed a document and that the signed information was not altered.

Proposed Non-Repudiation Framework

Examiner → Hash Document → Digital Signature → Timestamp → Certificate Validation → Secure Audit Log

1. Digital Signing

Before approving an examination document, the examiner's system should:

1. Generate a cryptographic hash of the document.
2. Sign the hash using the examiner's private key.
3. Attach the digital signature and certificate to the document.

The private key must be stored securely, preferably using a hardware security module or protected cryptographic device.

2. Timestamping

A trusted timestamp authority should provide a trusted time for the signature.

The timestamp should establish when the document was signed and prevent disputes such as claiming that a signature was created after the document was submitted.

3. Certificate Verification

The university should verify:

- Certificate validity period.
- Certificate chain to a trusted CA.
- Digital signature validity.
- Certificate key usage.
- Certificate revocation status using **CRL or OCSP**.
- Identity of the certificate holder.

A revoked or expired certificate should not automatically be accepted as valid.

4. Secure Audit Trail

Every significant event should be recorded, including:

- User identity.
- Document ID.
- Document hash.
- Signature information.
- Timestamp.
- Certificate serial number.
- Verification result.
- IP/device information where appropriate.
- Revocation-check result.
- Approval or rejection action.

The audit logs should be protected from unauthorized modification using access controls, cryptographic integrity protection, centralized logging, and preferably append-only or tamper-evident storage.

5. Signature Verification Process

Whenever an approval is reviewed:

Retrieve Document → Recalculate Hash → Verify Signature → Validate Certificate → Check Revocation → Validate Timestamp → Check Audit Log → Accept/Reject

6. Key Management

The university should implement:

- Strong private-key protection.
- Role-based access.
- Key rotation.
- Certificate renewal.
- Immediate revocation after compromise.
- Backup and recovery procedures.

Conclusion

This framework provides strong evidence of **who signed, what was signed, and when it was signed**. Combining digital signatures, trusted timestamps, certificate validation, revocation checking, and tamper-evident audit logs significantly reduces disputes and establishes reliable non-repudiation.

3. Kerberos Deployment Checklist for Smart-Metering IoT

Introduction

A smart-metering platform using Kerberos must carefully protect authentication tickets, keys, timestamps, and communication between IoT devices and authentication servers. Since IoT devices may have limited resources and may remain deployed for long periods, secure configuration and key management are critical.

Kerberos Deployment Checklist

1. Ticket Lifetime Policy

- Configure short-lived Ticket Granting Tickets and service tickets.
- Avoid unnecessarily long ticket lifetimes.
- Define maximum renewable lifetime.
- Immediately invalidate compromised credentials where possible.
- Require reauthentication for sensitive operations.

Short ticket lifetimes reduce the period during which stolen tickets can be abused.

2. Clock Synchronization Using NTP

Kerberos relies heavily on timestamps to prevent replay attacks.

The organization should:

- Synchronize all devices with trusted NTP servers.
- Configure an appropriate Kerberos clock-skew tolerance.
- Monitor synchronization failures.
- Prevent unauthorized devices from becoming trusted time sources.
- Use authenticated or otherwise secured time synchronization where appropriate.

3. Pre-Authentication

Kerberos pre-authentication should be enabled.

This prevents an attacker from easily requesting authentication material without first proving knowledge of the appropriate secret.

For sensitive environments, stronger mechanisms such as hardware-backed credentials or certificate-based pre-authentication can be considered.

4. Key Rotation

The organization should establish a formal key-management schedule:

- Rotate service-account keys periodically.
- Rotate device credentials according to risk.
- Immediately replace compromised keys.
- Protect long-term keys using secure storage.
- Avoid hard-coded credentials in firmware.
- Maintain secure key provisioning during device enrollment.

5. KDC Security

The Key Distribution Center is a critical component.

The organization should:

- Place KDC servers in a protected network.
- Restrict administrative access.
- Use redundancy for availability.
- Monitor authentication failures.
- Maintain secure backups.
- Log suspicious ticket requests.

5. Monitoring

Monitor:

- Repeated authentication failures.
- Unusual ticket requests.
- Authentication from unexpected devices.
- Clock synchronization failures.
- Abnormally long sessions.
- Service-account misuse.

Recommended Deployment Flow

IoT Device → Kerberos Authentication → KDC → TGT → Service Ticket → Metering Service → Continuous Monitoring

Conclusion

A secure Kerberos deployment requires more than simply enabling Kerberos. Short ticket lifetimes, accurate NTP synchronization, pre-authentication, secure key rotation, KDC protection, and continuous monitoring collectively reduce replay attacks, credential theft, and unauthorized access.

4. PKI Certificate Lifecycle Policy for an Ed-Tech Platform

Introduction

A Public Key Infrastructure provides trusted digital certificates for authenticating servers, users, devices, and services. Poorly managed X.509 certificates can result in expired certificates, unauthorized trust, weak keys, and security incidents. Therefore, the ed-tech platform should implement a complete certificate lifecycle policy.

Certificate Lifecycle

Certificate Request → Validation → Issuance → Deployment → Monitoring → Renewal → Revocation → Replacement

1. Certificate Issuance

Before issuing a certificate:

- Verify the identity of the requester.
- Verify domain or organizational ownership.
- Use approved certificate authorities.
- Generate strong private keys.
- Define appropriate key usage and extended key usage.
- Maintain an inventory of issued certificates.

Certificates should be issued only for approved purposes.

2. Secure Storage

Private keys must be protected using:

- Hardware security modules where appropriate.
- Access-control mechanisms.
- Encryption at rest.
- Restricted administrative permissions.

Private keys should never be unnecessarily shared between systems.

3. Renewal

The organization should monitor certificate expiration dates continuously.

Renewal should occur automatically before expiration rather than waiting until the final days of validity.

4. Revocation

Certificates should be revoked when:

- A private key is compromised.
- A certificate is issued incorrectly.
- The associated service is retired.
- Ownership changes.
- A certificate is no longer required.

Revocation status should be distributed through mechanisms such as **OCSP or CRLs**.

5. Monitoring

A centralized certificate inventory should track:

- Certificate owner.
- Domain/service.
- Issuer.
- Serial number.
- Expiration date.
- Key algorithm.
- Key size.
- Revocation status.

Automated alerts should identify certificates approaching expiration or using weak/obsolete configurations.

6. Automation Using ACME

The **ACME (Automatic Certificate Management Environment)** protocol can automate certificate issuance and renewal.

A typical workflow is:

ACME Client → Certificate Authority → Domain Validation → Certificate Issued → Automatic Deployment → Automatic Renewal

Automation reduces human errors such as:

- Forgetting certificate renewal.
- Deploying the wrong certificate.
- Missing expiration dates.
- Manually copying certificates.
- Using inconsistent configurations.

7. Trust Management

The organization should maintain an approved trust store and periodically remove unnecessary or untrusted CA certificates.

Conclusion

A formal certificate lifecycle policy combined with centralized monitoring and ACME automation provides scalable certificate management. It reduces certificate outages, human error, unauthorized trust, and security risks while maintaining reliable authentication across the ed-tech platform.

5. HMAC-SHA256 for a Stock Trading Platform

Introduction

A stock trading platform exchanges highly sensitive messages such as trade orders, confirmations, account instructions, and transaction details. Messages must be protected against modification and must be authenticated so that the receiver can verify that they originated from a party possessing the shared secret key.

Proposed HMAC Mechanism

The two trusted parties, **Party A and Party B**, first establish a secret key K .

For every message M :

Tag = HMAC-SHA256(K, M)

Party A sends:

Message + HMAC Tag

Party B receives the message, calculates HMAC-SHA256 using the same secret key, and compares the calculated tag with the received tag.

If both tags match, the message is accepted. Otherwise, it is rejected.

Message Flow

Party A

Message $\rightarrow$ HMAC-SHA256 + Shared Secret $\rightarrow$ Authentication Tag

↓

Message + Tag

↓

Party B

Received Message → HMAC-SHA256 + Shared Secret → Compare Tags

↓

Match = Accept

Mismatch = Reject

Protection Provided

HMAC-SHA256 provides:

1. **Message integrity** – Modification of the message causes the HMAC value to change.
2. **Message authentication** – Only a party possessing the shared secret should be able to generate a valid HMAC.
3. **Tamper detection** – Any alteration during transmission can be detected.

Why SHA-256?

SHA-256 is suitable because:

- It is a widely standardized cryptographic hash function.
- It has a 256-bit output.
- It has strong collision-resistance properties.
- It is widely implemented and supported.
- It provides a good balance between security and performance.

Key Length

A **256-bit randomly generated secret key** is an appropriate choice for a high-security trading environment.

The key should:

- Be generated using a cryptographically secure random generator.
- Never be transmitted in plaintext.
- Be stored using secure key-management infrastructure.
- Be rotated periodically.
- Be immediately replaced if compromise is suspected.
- Be unique to the appropriate communication relationship.

Additional Security Measures

For a real stock-trading system, HMAC should be combined with:

- TLS for confidentiality and transport protection.
- Sequence numbers or nonces to prevent replay.
- Message timestamps where appropriate.
- Strict key management.
- Transaction IDs for traceability.

- Secure audit logging.
- Rate limiting and anomaly detection.

Conclusion

HMAC-SHA256 provides a strong and efficient mechanism for authenticating messages and detecting tampering between trusted parties. A randomly generated 256-bit key, secure key storage, key rotation, replay protection, and TLS together provide a robust security architecture for stock-trading communications.